



WEB DESIGN PRINCIPLES
Final Term
Laboratory Activity Worksheet

Laboratory Activity #6: DOM and Control Structures

Student Name (LN, FN MI.):	Dagohoy, Justin John C.	Professor's Name:	Ms. Charlene Vergara
Student #:	202511717	Date of Submission:	3/23/2026

CILO#6: Demonstrate the use of Document Object Model with Conditional and Looping Statements.

Directions: Follow the instructions below.

1. Review the concepts in Document Object Model and Control Structures.
 - a. **The Document Object Model**
 - i. It represents your page as a tree of objects (elements, attributes, text). It is used to allow JavaScript to read, change, add, or delete parts of the page.
 - ii. Basic DOM Methods:

Attribute	Description
document.getElementById("id")	Selects an element by its ID
document.getElementsByClassName("class")	Selects elements by their class name
document.getElementsByTagName("tag")	Selects elements by tag name
document.querySelector("selector")	Selects the first matching element
document.querySelectorAll("selector")	Selects all matching elements

b. Conditional Statements

- i. The conditional statements allow the program to make decisions based on certain conditions. This means the code can choose between different actions depending on whether a condition is true or false. These statements help control the flow of the program, allowing for more dynamic behavior.

**ii. Conditional Statements Description and Syntax**

Statement	Description	Syntax
if	Tests 1 condition if true, executes default statement when true, exits the conditional statement if false.	<pre>if (condition) { // code }</pre>
if...else	Tests 1 condition if true, executes default statement when true, executes alternate statement the conditional statement if false.	<pre>if (condition) { // code } else { // code }</pre>
if...else if...else	Tests 2 or more conditions if true, executes the scoped statement of the true condition, executes alternate statement the conditional statement if false.	<pre>if (cond1) { // code } else if (cond2) { // code } else { // code }</pre>
switch	Executes one of many code blocks based on the value of an expression.	<pre>switch(expression) { case value: { // code break; } default: }</pre>

c. Looping Statements

- i. Looping statements are used to repeatedly execute a block of code as long as a specified condition is true. Looping allows you to perform repetitive tasks efficiently, without having to manually write the same code multiple times.

ii. Looping Statements Description and Syntax



Statement	Description	Syntax
for	Use when you know the number of iterations.	for (init; condition; update) { // code }
while	Use when you don't know the number of iterations but want to continue until a condition is false.	while (condition) { // code }
do...while	Executes the code block at least once before checking the condition.	do { // code } while (condition);
for...of (for arrays)	Loops through the properties of an object or the indices of an array.	for (let item of iterable) { // code }
for...in (for objects)	Loops through the values of an iterable object like arrays.	for (let key in object) { // code }

You can also open the provided PPTX for Module 6.

2. Write the given HTML code for `LA6_Surname.html`, and JavaScript code for `LabAct6_surname.js` that will produce the given output.



LabAct6.html

D: > ebooks > WDP > labAct > workSheets > New folder > LabAct6.html > html > head > style > li

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <!--Student Name: Last Name, First Name-->
4  <!--filename:LabAct6_surname.html-->
5  <head>
6      <meta charset="UTF-8">
7      <meta name="viewport" content="width=device-width, initial-scale=1.0">
8      <title>To-Do List with DOM, Conditional, Looping, and CSS</title>
9      <style>
10         /* Basic Styling */
11         body {
12             font-family: Arial, sans-serif;
13             margin: 20px;
14         }
15         ul {
16             list-style-type: none;
17             padding: 0;
18         }
19         li {
20             padding: 10px;
21             margin: 5px;
22             background-color: #f9f9f9;
23             border-radius: 5px;
24             display: flex;
25             align-items: center;
26         }
27         li.completed {
28             text-decoration: line-through;
29             color: gray;
30         }
```



```
31     li button {
32         margin-left: auto;
33         padding: 5px 10px;
34         background-color: #4CAF50;
35         color: white;
36         border: none;
37         border-radius: 5px;
38         cursor: pointer;
39     }
40     li button:hover {
41         background-color: #45a049;
42     }
43 </style>
44 </head>
45 <body>
46
47     <h2>To-Do List</h2>
48     <ul id="todoList"></ul>
49     <button id="addTaskBtn">Add New Task</button>
50
51     <script src="LabAct6_surname.js"></script>
52 </body>
53 </html>
54
```

LabAct6_surname.html



```
<> LabAct6.html  JS LabAct6_surname.js X
D: > ebooks > WDP > labAct > workSheets > New folder > JS LabAct6_surname.js > ...
1  //Student Name: Last Name, First Name
2  //filename:LabAct6_surname.js
3
4  // Comment 1:
5  let tasks = [
6    { task: "Buy groceries", completed: false },
7    { task: "Finish homework", completed: false },
8    { task: "Clean the house", completed: true },
9    { task: "Call Mom", completed: false }
10 ];
11
12 // Comment 2:
13 function displayTasks() {
14   // Get the <ul> element to add tasks to
15   const todoList = document.getElementById("todoList");
16
17   // Comment 3:
18   todoList.innerHTML = "";
19
20   // Comment 4:
21   for (let i = 0; i < tasks.length; i++) {
22     let li = document.createElement("li");
23
24     // Comment 5:
25     if (tasks[i].completed) {
26       li.classList.add("completed");
27     }
28
29     // Comment 6:
30     let taskText = document.createElement("span");
31     taskText.textContent = tasks[i].task;
32
```



```
33 // Comment 7:
34 let toggleButton = document.createElement("button");
35 toggleButton.textContent = tasks[i].completed ? "Undo" : "Complete";
36
37 // Comment 8:
38 toggleButton.addEventListener("click", function() {
39     tasks[i].completed = !tasks[i].completed; // Comment 9:
40     displayTasks(); // Comment 10:
41 });
42
43 // Comment 11:
44 li.appendChild(taskText);
45 li.appendChild(toggleButton);
46
47 // Comment 12:
48 todoList.appendChild(li);
49 }
50 }
51
52 // Comment 13:
53 document.getElementById("addTaskBtn").addEventListener("click", function() {
54     let newTask = prompt("Enter a new task: ");
55     if (newTask) {
56         tasks.push({ task: newTask, completed: false });
57         displayTasks(); // Comment 14:
58     }
59 });
60
61 // Comment 15:
62 displayTasks();
63
```

LabAct6_surname.js

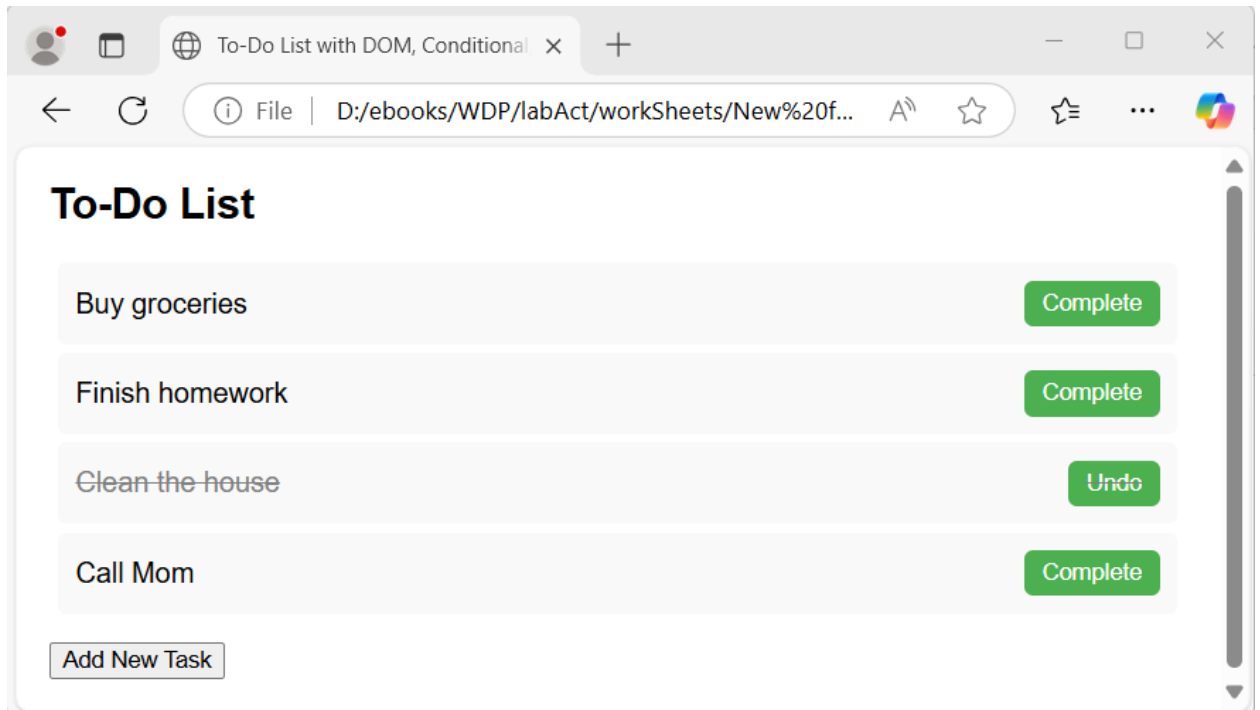


ADAMSON UNIVERSITY

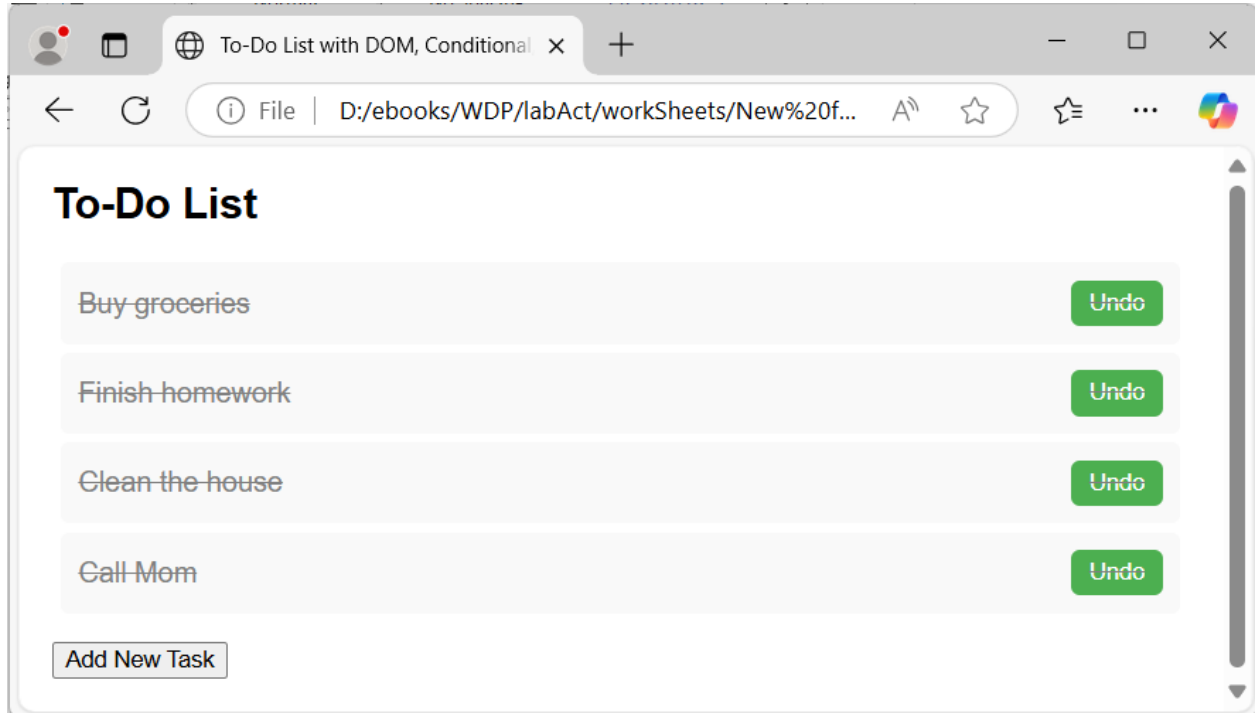
College of Computing and Information Technology

Information Technology and Information Systems Department

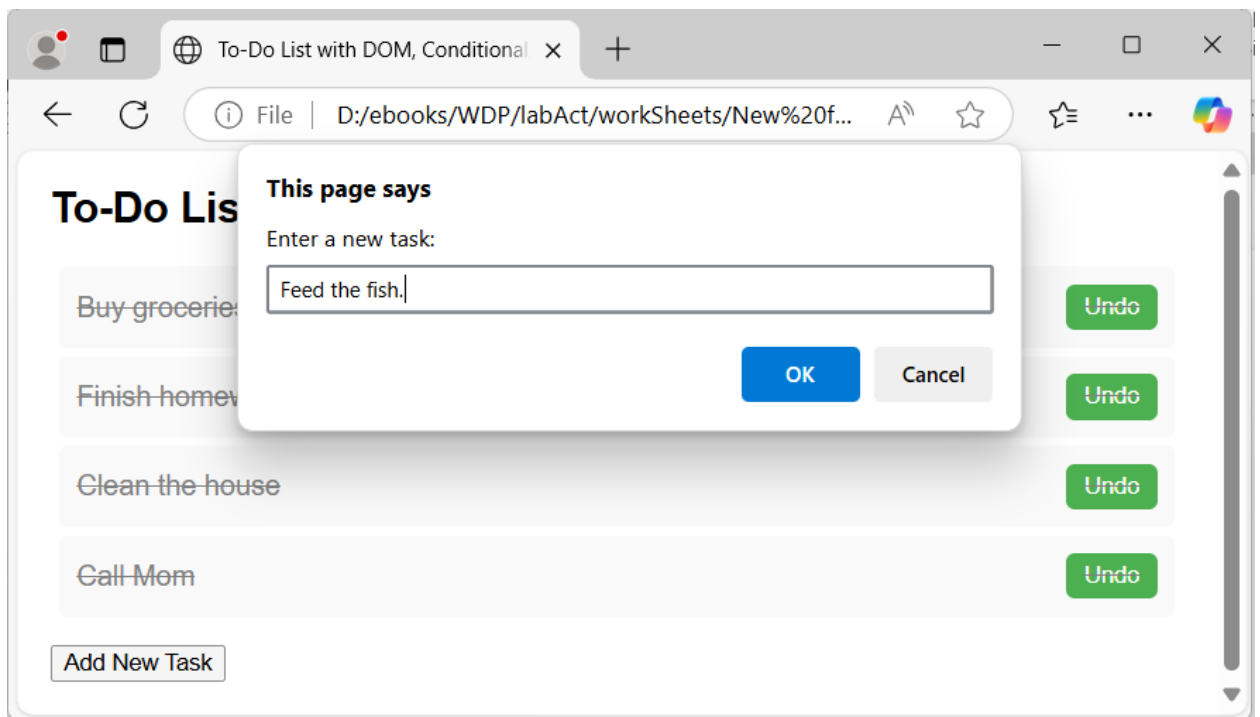
Ground Floor, St. Vincent De Paul Bldg., Adamson University, 900 San Marcelino St., Ermita, Manila 1000



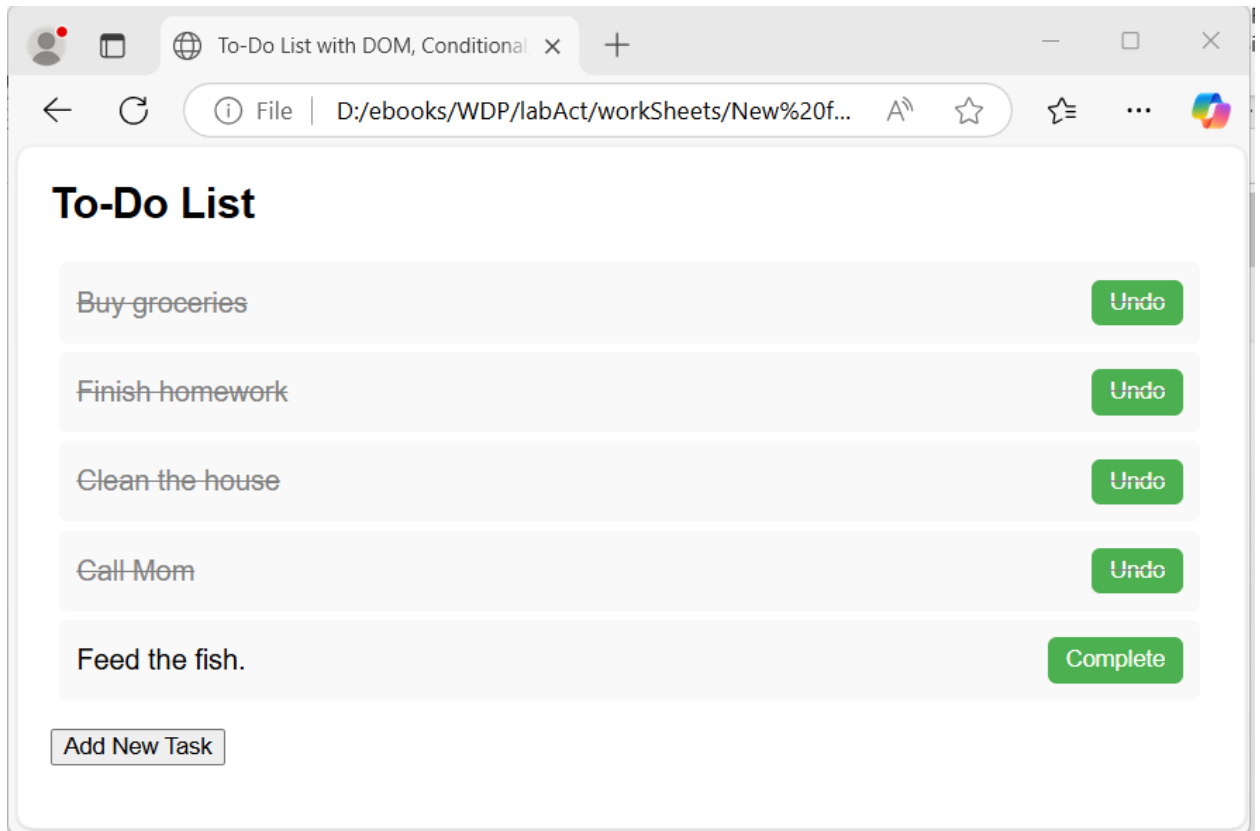
The default webpage upon loading in the web browser.



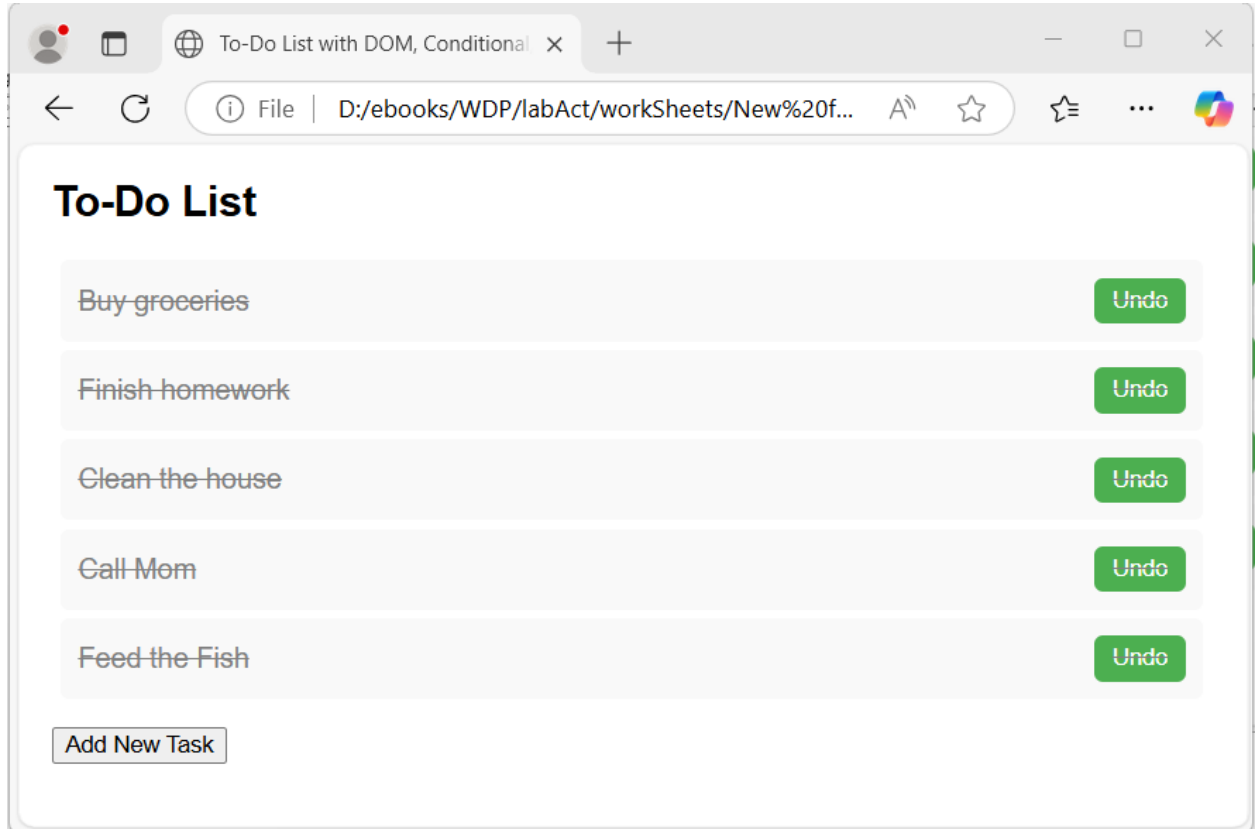
The webpage when the user clicks the Undo button.



The webpage when the user clicks the Add New Task button and enters a value in the dialog box.



The webpage after the user clicks the Ok button in the Dialog box.



The webpage when the user clicks the Undo button for Feed the Fish.

The codes should have:

- The JavaScript code is an external file of the HTML document.
- Comments to describe the code. Write the specific comment on the particular part of the JavaScript code as indicated by the phrase `//Comment 1: ,` `//Comment 2: ,` and so on.

3. Take screenshots of your complete codes and sample output generated from your code and paste it on the blank area below.
4. Save your LabAct Worksheet as LabAct6_Finals_surname.pdf. Upload these files in your OneDrive and share the link of each file to my email charlene.vergara@adamson.edu.ph with Editor Access.
5. Copy the file links and insert them as clickable links in BlackBoard Submission Area/Page as your submission content.
6. Your output will be graded based on the Rubrics Criteria found on page 10.



ADAMSON UNIVERSITY

College of Computing and Information Technology

Information Technology and Information Systems Department

Ground Floor, St. Vincent De Paul Bldg., Adamson University, 900 San Marcelino St., Ermita, Manila 1000



Place your Source Code Screenshots below: (Codes must be readable)



```
LA6_Dagohoy.html  JS LA6_Dagohoy.js X  WDP_LabAct #6 - Dagohoy.pdf
lab act 6 > JS LA6_Dagohoy.js > ...
1  // Student Name: Dagohoy, Justin John C.
2  // filename: LA6_Dagohoy.js
3
4  // Comment 1: List of tasks to show on the page
5  let tasks = [
6    { task: "Buy groceries", completed: false },
7    { task: "Finish homework", completed: false },
8    { task: "Clean the house", completed: true },
9    { task: "Call Mom", completed: false }
10 ];
11
12 // Comment 2: Function that draws the tasks on the screen
13 function displayTasks() {
14   // Get the <ul> element to add tasks to
15   const todoList = document.getElementById("todoList");
16
17   // Comment 3: Clear the list so it doesn't repeat old tasks
18   todoList.innerHTML = "";
19
20   // Comment 4: Loop through each task in the list
21   for (let i = 0; i < tasks.length; i++) {
22     let li = document.createElement("li");
23
24     // Comment 5: If task is done, add the "completed" style
25     if (tasks[i].completed) {
26       li.classList.add("completed");
27     }
28
29     // Comment 6: Create the text part of the task
30     let taskText = document.createElement("span");
31     taskText.textContent = tasks[i].task;
32
33     // Comment 7: Create a button that says "Undo" or "Complete"
34     let toggleButton = document.createElement("button");
35     toggleButton.textContent = tasks[i].completed ? "Undo" : "Complete";
36
37     // Comment 8: When the button is clicked, change the task status
38     toggleButton.addEventListener("click", function() {
39       tasks[i].completed = !tasks[i].completed;
40     });
41
42     li.appendChild(taskText);
43     li.appendChild(toggleButton);
44     todoList.appendChild(li);
45   }
46 }
```



```
lab act 6 > JS LA6_Dagohoy.js > ...
13  function displayTasks() {
27      }
28
29      // Comment 6: Create the text part of the task
30      let taskText = document.createElement("span");
31      taskText.textContent = tasks[i].task;
32
33      // Comment 7: Create a button that says "Undo" or "Complete"
34      let toggleButton = document.createElement("button");
35      toggleButton.textContent = tasks[i].completed ? "Undo" : "Complete";
36
37      // Comment 8: When the button is clicked, change the task status
38      toggleButton.addEventListener("click", function() {
39          tasks[i].completed = !tasks[i].completed; // Comment 9: Switch true to false or false to true
40          displayTasks(); // Comment 10: Redraw the list to show the change
41      });
42
43      // Comment 11: Put the text and button inside the list item
44      li.appendChild(taskText);
45      li.appendChild(toggleButton);
46
47      // Comment 12: Add the list item to the main <ul> tag
48      todoList.appendChild(li);
49  }
50  }
51
52  // Comment 13: Ask for a new task when the "Add" button is clicked
53  document.getElementById("addTaskBtn").addEventListener("click", function() {
54      let newTask = prompt("Enter a new task: ");
55      if (newTask) {
56          tasks.push({ task: newTask, completed: false });
57          displayTasks(); // Comment 14: Show the list with the new task added
58      }
59  });
60
61  // Comment 15: Run the function once when the page starts
62  displayTasks();
```



```
lab act 6 > <> LA6_Dagohoy.html > html > body
  2  <html lang="en">
  3  <head>
  7    <style>
19      li {
25        display: flex;
26        align-items: center;
27      }
28      li.completed {
29        text-decoration: line-through;
30        color: gray;
31      }
32
33      li button {
34        margin-left: auto;
35        padding: 5px 10px;
36        background-color: #4CAF50;
37        color: white;
38        border: none;
39        border-radius: 5px;
40        cursor: pointer;
41      }
42
43      li button:hover {
44        background-color: #45a049;
45      }
46    </style>
47  </head>
48  <body>
49
50    <h2>To-Do List</h2>
51    <ul id="todoList"></ul>
52    <button id="addTaskBtn">Add New Task</button>
53
54    <script src="LA6_Dagohoy.js"></script>
55  </body>
56 </html>
```



```
Welcome  <> LA6_Dagohoy.html X  JS LA6_Dagohoy.js
lab act 6 > <> LA6_Dagohoy.html > html > body
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>To-Do List with DOM, Conditional, Looping, and CSS</title>
7      <style>
8          /* Basic Styling */
9          body {
10             font-family: Arial, sans-serif;
11             margin: 20px;
12         }
13
14         ul {
15             list-style-type: none;
16             padding: 0;
17         }
18
19         li {
20             padding: 10px;
21             margin: 5px;
22             background-color: #f9f9f9;
23             border-radius: 5px;
24             display: flex;
25             align-items: center;
26         }
27
28         li.completed {
29             text-decoration: line-through;
30             color: gray;
31         }
32
33         li button {
34             margin-left: auto;
35             padding: 5px 10px;
36             background-color: #4CAF50;
37             color: white;
```



ADAMSON UNIVERSITY

College of Computing and Information Technology

Information Technology and Information Systems Department

Ground Floor, St. Vincent De Paul Bldg., Adamson University, 900 San Marcelino St., Ermita, Manila 1000



Place the Sample Output Screenshots below: (Must be visible and clear)

To-Do List

Buy groceries	Complete
Finish homework	Complete
Clean the house	Complete
Call Mom	Complete
Add New Task	

To-Do List

Buy groceries	Undo
Finish homework	Undo
Clean the house	Undo
Call Mom	Undo
Make Banana Bread!	Undo
Add New Task	

To-Do List

Buy groceries	Undo
Finish homework	Undo
Clean the house	Undo
Call Mom	Undo
Make Banana Bread!	Complete
Add New Task	



To-Do List

Buy-groceries

Finish-homework

Clean-the-house

Call-Mom

Add New Task

This page says

Enter a new task:

Make Banana Bread!

OK

Cancel

Undo

Undo

Undo

Undo

To-Do List

Buy-groceries

Finish-homework

Clean-the-house

Call-Mom

Add New Task

Undo

Undo

Undo

Undo

Rubrics Criteria

Criteria	Excellent 100%	Satisfactory 75%	Developing 50%	Needs Improvement 25%
Accuracy of Comments 30%	Comments correctly describe what the code does and why it's needed for	Minor inaccuracies, but the overall logic is understandable.	Some comments are incorrect or too general.	Comments are mostly wrong, confusing, or absent.



	DOM, loops, and conditionals.			
Coverage of Key Code Areas 20%	All important sections (DOM manipulation, loops, conditionals) are well-commented.	Most major sections are commented; some minor omissions.	Some key sections lack comments or are not well explained.	Few or no relevant sections are commented.
Clarity and Readability 20%	Comments are clear, concise, and easy to read.	Mostly clear with a few awkward or unclear phrases.	Some comments are confusing or poorly written.	Comments are hard to understand or poorly structured.
Relevance and Usefulness 20%	Comments enhance understanding without stating the obvious (e.g., don't just restate the code).	Generally useful, with minor cases of redundancy.	Comments are sometimes redundant or not helpful.	Comments are unnecessary, irrelevant, or not useful.
Integration with Code 10%	Comments are well-placed and consistently formatted with the code structure.	Mostly well-placed with minor formatting or spacing issues.	Inconsistent placement or formatting makes reading harder.	Poorly integrated; comments are scattered or disruptive.